

Módulo 11 – Capítulo 05 – Sección 01

LLMOps vs MLOps tradicional: qué cambia cuando el modelo es un LLM

El MLOps tradicional resuelve un problema bien definido: gestionar el ciclo de vida de modelos de machine learning cuyos artefactos principales son archivos de parámetros (.pkl, .pt, .onnx), cuya calidad se mide con métricas objetivas (accuracy, F1, AUC-ROC calculadas contra etiquetas verdaderas), y cuyas actualizaciones requieren un proceso de reentrenamiento con nuevos datos. Este problema está bien resuelto por herramientas maduras como MLflow, Weights & Biases, y DVC. LLMOps no reemplaza MLOps sino que lo extiende para acomodar propiedades de los LLMs que invalidan algunas de las suposiciones centrales del MLOps clásico.

La primera diferencia crítica es la **evaluación**. En MLOps, la calidad del modelo se mide comparando sus predicciones contra etiquetas verdaderas — un proceso automático, barato, y determinista. En LLMOps, la calidad de un LLM en tareas de generación de texto es multidimensional y no determinista: coherencia, relevancia, precisión factual, alineación con el estilo esperado, y ausencia de contenido problemático son dimensiones que frecuentemente requieren evaluadores LLM-as-a-judge (usando GPT-4 o Claude para calificar respuestas de manera automática) o evaluaciones humanas. Ambos enfoques son más lentos y costosos que las métricas automáticas del MLOps tradicional, y requieren un diseño de pipeline de evaluación más cuidadoso para ser ejecutables como parte del pipeline de CI/CD sin añadir horas de latencia a cada despliegue.

La segunda diferencia es el **versionado**. En MLOps, el artefacto principal que se versiona es el modelo. En LLMOps, el artefacto que cambia con mayor frecuencia no es el modelo — que puede mantenerse estable durante meses — sino el **prompt**: el system prompt, las instrucciones de formato, el texto de los few-shots examples. Un cambio de prompt puede transformar radicalmente el comportamiento del sistema sin modificar ningún parámetro del modelo. Sin un sistema de versionado de prompts tan riguroso como el versionado de código — con historial, testing antes del despliegue, y capacidad de rollback — el equipo opera sin control sobre uno de los principales determinantes del comportamiento del sistema.

La tercera diferencia es el **costo de inferencia**. Un modelo de clasificación en producción puede procesar miles de inferencias por segundo en una CPU estándar a un costo de fracciones de centavo. Un LLM de 70B parámetros en GPU puede costar 0.01-0.10 USD por inferencia, y a escala de millones de inferencias al día, ese costo se convierte en la restricción financiera que determina la viabilidad del proyecto. La optimización de costos de inferencia — mediante routing de modelos, semantic caching, compresión de prompts — es una responsabilidad central del equipo de LLMOps que no tiene equivalente en el MLOps tradicional.

La cuarta diferencia, más sutil pero igualmente importante, es el **drift de comportamiento sin drift de datos**. En MLOps, la degradación del modelo generalmente se explica por cambios en la distribución de los datos de entrada (concept drift o data drift). En LLMOps, un LLM puede degradar su calidad de respuesta sin ningún cambio en los datos de entrada: el proveedor puede actualizar el modelo en la misma versión de API sin anuncio previo, cambiando sutilmente el comportamiento del sistema. Este tipo de drift es invisible para los sistemas de monitoreo de MLOps tradicionales y requiere evaluación continua de la calidad de las respuestas generadas.

Diferencias técnicas entre LLMOps y MLOps

- **Evaluación no determinista:** los LLMs producen respuestas distintas con el mismo input (temperatura > 0), lo que invalida las suites de testing basadas en igualdad exacta y requiere evaluadores semánticos o LLM-as-a-judge calibrados.
- **Prompt como artefacto de ingeniería:** los prompts deben versionarse en un prompt registry con changelog, tests de regresión contra el golden set, y proceso de revisión por pares antes de desplegar a producción.
- **Costo de fine-tuning vs actualización de prompts:** re-entrenar un LLM de 7B parámetros con LoRA cuesta entre 100 y 1.000 USD en GPU cloud; actualizar un prompt cuesta centavos — la decisión de cuándo hacer fine-tuning vs prompt engineering es una decisión económica tanto como técnica.
- **Inferencia a escala con vLLM:** despliegue de modelos open-source con vLLM usando PagedAttention para maximizar el throughput de tokens por segundo, reduciendo el número de GPUs necesarias para la misma carga en 2-4x.
- **Drift de comportamiento por actualización del proveedor:** un LLM puede degradar su calidad de respuesta no por cambios en los datos de entrada sino por actualizaciones silenciosas del modelo por parte del proveedor, requiriendo monitoreo continuo de la calidad de las respuestas en producción.

Para recordar: LLMOps requiere construir el sistema de evaluación continua antes de desplegar el primer modelo a producción — sin evaluación automatizada, no hay manera de detectar regresiones cuando el sistema cambia, ya sea por cambios de prompt, de modelo, o por actualizaciones del proveedor.

La sección siguiente profundiza en el componente central del sistema de evaluación continua: los tres tipos de evaluadores, el golden dataset como fundación, y la integración en el pipeline de CI/CD como gate de despliegue.

Módulo 11 – Capítulo 05 – Sección 02

Evaluación continua de LLM en producción: calidad, coherencia y alineación con el negocio

La evaluación de sistemas de LLM es el componente de LLMOps que más frecuentemente se subestima en las fases tempranas y que más se lamenta en los incidentes tardíos. Un equipo que despliega un sistema de IA sin evaluación automatizada opera a ciegas: sabe que el sistema respondía bien durante el piloto, pero no tiene forma de detectar cuándo una actualización del prompt, un cambio en el modelo, o una modificación en los datos de RAG produce una degradación de calidad. La primera señal de que algo está mal llega de un usuario insatisfecho, días después del cambio que introdujo el problema.

La evaluación continua debe operar en tres planos simultáneamente. El primero es la **calidad técnica**: coherencia gramatical, seguimiento de instrucciones del prompt, ausencia de alucinaciones verificables. El segundo es la **alineación con el negocio**: las respuestas representan correctamente la política de la empresa, usan el tono adecuado para el contexto, y contienen la información que el usuario necesita para tomar una decisión o resolver su problema. El tercero es la **seguridad y cumplimiento**: ausencia de contenido prohibido, no divulgación de información confidencial, respeto a las restricciones configuradas por el tenant. Estos tres planos tienen métricas distintas, evaluadores distintos, y thresholds de alerta distintos.

La implementación técnica combina tres tipos de evaluadores con diferente costo y precisión. Los **evaluadores basados en reglas** son los más rápidos y baratos: verificaciones deterministas como "la respuesta está en el idioma correcto", "la respuesta contiene una referencia de ticket cuando se solicita", o "la longitud de la respuesta está dentro del rango configurado". Se ejecutan en el path síncrono de cada petición con latencia adicional de menos de 10ms. Los **evaluadores basados en embeddings** miden la similitud semántica entre la respuesta generada y las respuestas de referencia del golden set: una similitud coseno por debajo del threshold (típicamente 0.85) indica que la respuesta se desvía semánticamente del comportamiento esperado. Los **evaluadores LLM-as-a-judge** son los más costosos y precisos: un LLM de alta calidad (GPT-4o o Claude Opus) recibe la pregunta del usuario, la

respuesta del sistema, y un rubric de evaluación, y califica la respuesta en dimensiones como relevancia (1-5), precisión factual (1-5), y adherencia al estilo (1-5). Se ejecutan de manera asíncrona sobre el 5-10% del tráfico de producción para no impactar la latencia.

La integración en el pipeline de CI/CD como **gate de despliegue** es el elemento que convierte la evaluación de un proceso de calidad en un mecanismo de control de cambios. Cada pull request que modifica el sistema prompt, actualiza el modelo, o cambia la configuración del RAG debe pasar el gate de evaluación antes de poder fusionarse a la rama de producción: el pipeline ejecuta el golden dataset completo, calcula las métricas de evaluación, y rechaza el merge si cualquier métrica cae por debajo del threshold definido. Esta integración transforma la evaluación de un proceso periódico en una propiedad continua del sistema.

Nota del Arquitecto: La calibración de los evaluadores LLM-as-a-judge es el trabajo más importante y menos glamuroso de LLMOps. Un evaluador LLM-as-a-judge que no está calibrado contra juicio humano puede producir scores que no correlacionan con la satisfacción real del usuario — y en ese caso, el gate de CI/CD está midiendo algo diferente a lo que el usuario experimenta. Dedicar 2-3 semanas a calibrar el evaluador contra 200-300 casos con evaluación humana como ground truth es la inversión que hace que el sistema de evaluación sea fiable.

Componentes del sistema de evaluación continua

- **Golden dataset:** conjunto de 100-500 pares (input, output_esperado) representativos de los casos de uso críticos, mantenido en Git con proceso de revisión para agregar nuevos casos cuando se identifican brechas de cobertura.
- **Evaluadores LLM-as-a-judge:** prompts de evaluación calibrados con correlación verificada contra evaluación humana, que califican dimensiones como relevancia, precisión factual, y adherencia al estilo con scores de 1-5 reproducibles.
- **Métricas de evaluación de RAG con RAGAS:** faithfulness (la respuesta es fiel al contexto recuperado), answer relevancy (la respuesta es relevante a la pregunta), y context precision (el contexto recuperado es relevante para la pregunta), ejecutadas de manera asíncrona sobre el tráfico de producción.
- **CI/CD gate de calidad:** job en GitHub Actions o GitLab CI que ejecuta la suite de evaluación sobre el golden dataset, bloquea el merge si las métricas caen por debajo del threshold, y genera un reporte de evaluación detallado visible en el pull request.

- **Sampling de producción:** interceptor en el servicio de orquestación que con probabilidad configurable (1-5%) registra pares (input, output) en una cola asíncrona para evaluación posterior, construyendo el dataset de producción que alimenta la mejora continua del golden set.
-

Buena práctica: Las métricas de evaluación y sus thresholds deben definirse antes de desplegar el primer sistema de IA a producción, junto con el proceso de revisión cuando una métrica falla — ¿rollback automático o revisión humana? La respuesta a esa pregunta determina la arquitectura del sistema de evaluación.

Con el sistema de evaluación continua establecido como pilar del LLMOps, la sección siguiente aborda el artefacto que más frecuentemente se gestiona de manera informal y que más frecuentemente produce incidentes cuando no se gestiona correctamente: el prompt.

Módulo 11 – Capítulo 05 – Sección 03

Prompt versioning y prompt registry: gestionar prompts como artefactos de ingeniería

En el ecosistema de LLMOps, los prompts ocupan un lugar análogo al que ocupa el código fuente en el desarrollo de software: son el principal determinante del comportamiento del sistema, cambian frecuentemente a medida que el equipo refina el sistema, pueden introducir regresiones si se modifican sin testing, y su historial es crítico para entender por qué el sistema se comporta de una manera específica en un momento dado. Sin embargo, en la mayoría de los equipos que empiezan con LLMs, los prompts se gestionan como strings hardcodeados en archivos de configuración, variables de entorno, o directamente en el código fuente. Esta práctica produce consecuencias específicas: cuando aparece un incidente de calidad en producción a las 2am, nadie puede determinar qué versión del prompt estaba activa cuando ocurrió el problema ni comparar el comportamiento con la versión anterior.

El **prompt registry** es el componente de infraestructura que resuelve este problema. En su forma más simple, un prompt registry es una tabla de base de datos (PostgreSQL con JSONB funciona perfectamente para comenzar) que almacena cada versión de cada prompt con su identificador único, su versión semántica, su metadata (autor, fecha, modelo objetivo, parámetros de inferencia recomendados), y el resultado de la evaluación contra el golden set. El servicio de orquestación llama al prompt registry al inicio de cada petición para obtener el prompt activo para el caso de uso correspondiente, o lo consulta con TTL corto en Redis para reducir la latencia. Este diseño habilita una capacidad crítica: modificar el prompt en producción — y hacer rollback si es necesario — sin desplegar código nuevo.

El **versionado semántico de prompts** aplica las mismas convenciones del versionado de código: una versión **major** (v2.0.0) indica un cambio de estructura que puede producir comportamientos cualitativamente distintos, una versión **minor** (v1.1.0) indica un refinamiento que mejora el comportamiento dentro de la misma estructura, y una versión **patch** (v1.0.1) indica una corrección menor de un error específico. Esta convención permite a los equipos comunicar con precisión la magnitud del cambio y calibrar la extensión del testing

requerido: un patch puede pasar con el golden set existente; un mayor puede requerir la construcción de nuevos casos en el golden set y una revisión más exhaustiva.

La capacidad de **canary deployment de prompts** es la característica de mayor valor en producción: desplegar una nueva versión de prompt al 5-10% del tráfico mientras la versión anterior continúa sirviendo el 90-95% restante, monitoreando las métricas de calidad en tiempo real antes de completar el rollout. Esta capacidad convierte el despliegue de un prompt nuevo de un evento de riesgo alto (¿cómo se comporta en producción con tráfico real?) en un proceso de validación controlada con datos reales y con capacidad de rollback en segundos. Herramientas gestionadas como LangSmith y Langfuse soportan esta capacidad de manera nativa; un sistema propio puede implementarla con la combinación del prompt registry en PostgreSQL y el sistema de feature flags.

La conexión con el API management del Capítulo 02 es directa: el prompt registry aplica los mismos principios de versionado semántico descritos allí, pero al artefacto específico del prompt en lugar de al contrato de API. Un cambio mayor en el prompt equivale a un cambio breaking en la API: requiere revisión, período de validación, y una política clara de cuánto tiempo conviven versiones mayor distintas.

Aspectos técnicos del prompt registry

- **Versionado semántico de prompts:** `prompt_id` + versión semántica (major.minor.patch), con convención que major = cambio de estructura, minor = refinamiento dentro de la misma estructura, patch = corrección de error específico.
- **Metadata y trazabilidad:** cada versión registra autor, fecha, modelo objetivo y versión, parámetros de inferencia recomendados, resultado de evaluación contra golden set, y notas de changelog que documentan qué cambió y por qué.
- **Canary prompts:** despliegue de nueva versión al 5-10% del tráfico con monitoreo de métricas de calidad en tiempo real antes de completar el rollout al 100%, con rollback automático si las métricas degradan más del threshold configurado.
- **Prompt templates con variables:** prompts parametrizados con Jinja2 o Python f-strings tipadas que reciben variables de contexto en tiempo de ejecución, separando la estructura del prompt (estable) del contenido dinámico (variable).
- **Audit trail de cambios:** log inmutable de qué versión de prompt generó cada respuesta en producción, indexado por timestamp y `conversation_id`, crítico para investigar incidentes de calidad o compliance con evidencia verificable.

Principio rector: Gestionar prompts como código — con versionado en Git, revisión por pares, testing automatizado contra el golden set, y despliegue controlado con canary — es la diferencia entre un equipo que puede operar sus sistemas de IA con confianza y uno que opera en modo reactivo permanente, descubriendo los problemas a través de las quejas de los usuarios.

La sección siguiente profundiza en el mecanismo que permite comparar versiones de forma estadísticamente rigurosa: el A/B testing de modelos y prompts en producción enterprise.

Módulo 11 – Capítulo 05 – Sección 04

A/B testing de modelos y prompts en producción enterprise

La pregunta "¿es esta versión del prompt mejor que la anterior?" no puede responderse adecuadamente con una revisión manual de ejemplos ni con la intuición del equipo. Puede responderse con evidencia estadística: un experimento A/B que asigna tráfico real a las dos versiones, mide las métricas de calidad relevantes en condiciones de producción idénticas, y determina con un nivel de confianza definido si la diferencia observada es estadísticamente significativa o podría explicarse por varianza aleatoria. El A/B testing en sistemas de LLM enterprise tiene particularidades técnicas que lo diferencian del A/B testing web convencional, y que si se ignoran producen resultados espurios que llevan a decisiones erróneas.

La primera particularidad es la **consistencia de la asignación de variante**. En un test A/B de click-through rates, cada usuario puede experimentar una variante distinta en cada visita sin contaminar el experimento. En un sistema de chat o de asistente conversacional, cambiar la variante entre mensajes de la misma sesión contamina el experimento: si el usuario recibe la variante A en un mensaje y la variante B en el siguiente, no puede atribuirse su satisfacción o insatisfacción a ninguna de las dos. La asignación debe ser consistente por usuario o por sesión, implementada mediante hash del `user_id` o del `session_id` con módulo sobre el porcentaje de tráfico de cada variante. Un framework de feature flagging (LaunchDarkly, Unleash) gestiona esta consistencia de manera automática.

La segunda particularidad es el **tamaño de muestra necesario**. En A/B tests de click-through rates, los efectos son grandes (5-20% de diferencia) y la varianza es baja, lo que permite detectar diferencias con muestras de cientos o miles de sesiones. En A/B tests de calidad de LLM, los efectos son más pequeños (1-5% de diferencia en quality score) y la varianza es alta (los LLMs producen respuestas distintas para el mismo input), lo que requiere típicamente 1.000 a 5.000 conversaciones por variante para detectar diferencias con potencia estadística del 80% a nivel de significancia del 5%. En sistemas de uso moderado, esto puede representar días o semanas de tráfico de producción — el experimento debe planificarse con ese horizonte temporal.

Los **tests de modelos** son especialmente valiosos porque permiten tomar decisiones de selección de modelo basadas en datos de negocio reales en lugar de en benchmarks públicos. GPT-4o puede superar a Claude Sonnet en MMLU mientras Claude Sonnet produce mejores resultados en el caso de uso específico de análisis de contratos legales de la empresa, porque los benchmarks públicos no capturan las particularidades del dominio, el vocabulario, y los tipos de documentos que el sistema debe procesar. Un A/B test de 2 semanas con 2.000 conversaciones reales por variante produce más evidencia de decisión que cualquier benchmark público.

Nota del Arquitecto: El análisis de resultados de A/B tests de LLM requiere más cuidado estadístico que el de tests web. Las métricas de quality score (1-5) no son normales — tienden a tener distribuciones bimodales o con colas gruesas — lo que invalida el uso del t-test simple para calcular la significancia estadística. Usar bootstrapping para calcular intervalos de confianza produce resultados más robustos independientemente de la distribución subyacente de la métrica.

Aspectos técnicos del A/B testing en LLMOps

- **Asignación de variante consistente:** hash determinista del user_id o session_id módulo 100 para asignar porcentajes exactos de tráfico, con el resultado registrado en cada petición para permitir segmentación del análisis posterior por variante.
- **Métricas de evaluación A/B:** combinación de métricas automáticas (quality score LLM-as-a-judge, similarity score contra golden set) y métricas de negocio (thumbs up/down del usuario, tasa de follow-up de aclaración, tiempo hasta resolución del problema).
- **Multi-armed bandit como alternativa:** algoritmos Thompson Sampling o Epsilon-Greedy que asignan más tráfico a la variante ganadora dinámicamente durante el experimento, reduciendo el costo de exponer a los usuarios a variantes de menor calidad — a cambio de mayor complejidad estadística en la interpretación.
- **Análisis de resultados con bootstrapping:** para métricas no normales (scores discretos de 1-5), usar bootstrapping con 10.000 resamples para calcular intervalos de confianza del 95%, en lugar de t-tests que asumen normalidad de la distribución.
- **Logging estructurado de experimentos:** registrar en cada petición el experiment_id, la variante asignada, el model_id, el prompt_version, y todas las métricas calculadas en formato JSON estructurado para análisis posterior en BigQuery o ClickHouse.

Buena práctica: Nunca ejecutar más de un experimento simultáneo que afecte la misma métrica de negocio en el mismo segmento de usuarios. La interferencia entre experimentos — cuando la variante del experimento A afecta la métrica que mide el experimento B — produce resultados que parecen estadísticamente válidos pero que reflejan la interacción entre experimentos, no el efecto de ninguna de las variantes individuales.

La sección siguiente cierra el ciclo del LLMOps con el componente más crítico en producción y el más frecuentemente omitido del plan de despliegue: las estrategias de rollback cuando una actualización del modelo o del prompt degrada la calidad del sistema.

Módulo 11 – Capítulo 05 – Sección 05

Rollback de LLM: estrategias para revertir cuando la actualización del modelo degrada calidad

El plan de rollback de LLM es el componente del ciclo de vida del sistema que se documenta con menor rigor y que más se necesita cuando todo lo demás ha fallado. En la mayoría de los equipos, el plan de rollback existe como una sección al final del documento de despliegue que dice algo como "si hay problemas, revertir al modelo anterior". Esta descripción es insuficiente: no especifica qué constituye "problemas" (¿qué threshold de degradación activa el rollback?), no define el mecanismo técnico (¿cómo se cambia el modelo activo — despliegue de código, cambio de configuración, feature flag?), no indica cuánto tiempo toma el rollback en condiciones reales, y no documenta si el plan fue probado antes de que se necesitara en producción.

La primera capa del plan de rollback — y la más simple de implementar — es mantener el modelo anterior disponible y recuperarlo en la configuración del servicio de orquestación sin despliegue de código nuevo. Esto requiere que dos condiciones se cumplan desde el diseño: el endpoint de modelo activo debe ser un valor de configuración (feature flag, variable en el config service) que puede modificarse en tiempo real sin reiniciar el servicio, y el modelo anterior debe seguir siendo accesible — lo que no siempre está garantizado si el proveedor deprecia versiones periódicamente.

La segunda capa, más robusta, implementa el **blue-green deployment para LLMs**: mantener dos versiones del servicio de inferencia activas simultáneamente (blue: versión actual en producción, green: versión nueva en validación), con el router de modelos enviando el 100% del tráfico a blue durante el período de validación de green. Cuando se decide promover green a producción, el router cambia el 100% del tráfico a green. Si se detecta una degradación, el rollback es instantáneo: el router vuelve a enviar el 100% a blue. El tiempo de rollback en este modelo es de segundos — el tiempo de propagación del cambio de configuración del router, no el tiempo de despliegue de una nueva imagen Docker.

El **canary rollout con rollback automático** es el mecanismo más sofisticado: desplegar la nueva versión al 5% del tráfico con un sistema de monitoreo que evalúa continuamente las métricas de calidad de la variante nueva versus la actual, y revierte automáticamente al 0% si las métricas de la variante nueva caen más del threshold configurado (por ejemplo, 5% de degradación en el LLM-as-a-judge score promedio de las últimas 100 peticiones). El rollback automático convierte la validación de nuevas versiones de modelos en un proceso continuo y sin riesgo: los usuarios que experimentan la variante nueva están siendo protegidos por el sistema de monitoreo que revertirá la exposición antes de que la degradación sea significativa.

Un aspecto crítico del rollback de LLMs que no existe en el rollback de modelos ML tradicionales es la relación entre el modelo y el prompt. Los prompts están frecuentemente optimizados para la versión específica del modelo con la que fueron desarrollados: un prompt diseñado y refinado para GPT-4 puede no producir el mismo comportamiento con GPT-4o, y viceversa. El plan de rollback del modelo debe incluir el rollback simultáneo al prompt asociado a esa versión del modelo — el prompt registry debe mantener el vínculo entre cada versión de prompt y la versión de modelo para la que fue optimizada.

Estrategias técnicas de rollback

- **Blue-green deployment para LLMs:** dos versiones del servicio de inferencia activas simultáneamente con conmutación instantánea del router, sin tiempo de despliegue en el rollback — solo tiempo de propagación del cambio de configuración.
- **Canary rollout con rollback automático:** despliegue al 5% del tráfico con monitoreo continuo y rollback automático si las métricas de calidad caen más del threshold, convirtiendo el rollout en un proceso de validación continua con datos reales.
- **Feature flag para versión de modelo:** el modelo activo configurado mediante feature flag (no hardcodeado), con rollback instantáneo modificando una variable de configuración desde el dashboard del sistema de feature flags.
- **Snapshot de prompts asociados al modelo:** cuando se hace rollback del modelo, también se revierte automáticamente al prompt optimizado para esa versión, preservando la coherencia entre el modelo y la estrategia de prompting.
- **Runbook de rollback documentado y practicado:** procedimiento paso a paso con tiempos esperados por etapa (objetivo: rollback completo en menos de 10 minutos), responsables identificados, y criterios de éxito verificables, ejecutado como fire drill trimestral.

Para recordar: Un plan de rollback que no se ha practicado no es un plan — es una esperanza. Los fire drills regulares de rollback de LLM son tan importantes en LLMOps como los game days en Site Reliability Engineering: revelan los pasos que faltan, los tiempos que se subestimaron, y las dependencias que nadie documentó.

El cierre del capítulo integra los cinco componentes de LLMOps en una visión de sistema: por qué la infraestructura de LLMOps es la diferencia entre un equipo que puede operar sus sistemas de IA con confianza y uno que opera en modo reactivo permanente.

Módulo 11 – Capítulo 05 – Sección 06

Cierre: LLMOps es MLOps adaptado a la naturaleza no determinista de los modelos de lenguaje

LLMOps no reinventa MLOps sino que lo extiende para acomodar las propiedades únicas de los modelos de lenguaje que invalidan algunas de las suposiciones centrales del MLOps clásico. El no determinismo de las respuestas, la imposibilidad de evaluar la calidad con métricas simples, el costo elevado de inferencia que convierte la optimización en una responsabilidad de primer nivel, la dependencia de proveedores externos que pueden actualizar o deprecitar modelos sin previo aviso suficiente, y el prompt como artefacto de ingeniería con su propio ciclo de vida — ninguna de estas propiedades tiene un equivalente directo en el MLOps de modelos supervisados clásicos.

Los componentes de LLMOps cubiertos en este capítulo — la evaluación continua con LLM-as-a-judge y golden datasets, el prompt registry versionado con canary deployments, el A/B testing con asignación consistente por sesión y análisis estadístico apropiado, y las estrategias de rollback con blue-green deployments y feature flags — constituyen la infraestructura mínima para operar sistemas de LLM en producción enterprise con confianza. No son opcionales: un equipo que opera sin esta infraestructura puede funcionar adecuadamente durante los primeros meses gracias a la calidad del modelo subyacente, pero cuando el proveedor actualiza el modelo, cuando el caso de uso evoluciona, o cuando el volumen de usuarios aumenta sustancialmente, la ausencia de LLMOps se convierte en la causa principal de incidentes de calidad que dañan la reputación del sistema y la confianza de los stakeholders internos.

La inversión en LLMOps también es la que habilita la mejora continua del sistema: sin evaluación automatizada, no hay manera de medir el impacto de cada cambio; sin A/B testing, las mejoras se deciden por intuición en lugar de por evidencia; sin prompt registry, los cambios de prompts no tienen historia; sin rollback planificado, cada mejora es un riesgo. La infraestructura de LLMOps transforma el ciclo de mejora del sistema de IA de un proceso episódico y arriesgado en un proceso continuo y controlado.

El siguiente capítulo aplica los principios de LLMOps al caso de uso más frecuente en enterprise: los sistemas de RAG que conectan los LLMs con el conocimiento corporativo. La evaluación de sistemas RAG introduce métricas específicas del dominio (faithfulness, context precision, answer relevancy), y el ciclo de vida del knowledge management añade una dimensión de frescura del índice vectorial que no existe en los sistemas de LLM sin recuperación.

"La diferencia entre investigación y producción en machine learning es que en investigación se maximiza el rendimiento del modelo, mientras que en producción se maximiza la confiabilidad del sistema que lo rodea." — Chip Huyen, autora de "Designing Machine Learning Systems" y "AI Engineering"